

# AR Backend Database Schema Documentation

## 1. Core Architectural Principles

Before diving into the specific tables, it is crucial to understand the global patterns applied across this schema:

- **Multi-Tenancy (Shared Schema):** Almost every table in the database contains an `org_id` column (e.g., `ste_org_id`, `cou_org_id`). This ensures strict tenant data isolation. Every backend query must filter by this ID to prevent cross-tenant data leaks.
  - **Soft Deletes:** Records are rarely hard-deleted. Instead, they use an `ended_at` DATETIME column. If `ended_at` is NULL, the record is active; if populated, it is considered deleted.
  - **Auditability:** Every major table includes `created_at` and `updated_at` columns to track when records are created and modified.
- 

## 2. Entity Modules & Relationships

### Module 1: Global Settings

These tables are tenant-agnostic and apply system-wide.

- **subscription\_plan:** Defines the available billing plans (MONTHLY, YEARLY), pricing, and limits on course creation.
- **role:** A lookup table defining the access control levels (ORG\_ADMIN, COURSE\_EDITOR, SUPPORT, STUDENT) and their descriptions.

### Module 2: Tenant Core (Organisation Management)

This module handles the businesses (tenants) that use the platform and their staff.

- **organisation:** The root tenant record storing owner details, contact information, and login credentials.
- **organisation\_subscription:** A joining table that links an `organisation` to a `subscription_plan`, tracking the active status of their billing.
- **profile & address:** One-to-one extensions of the `organisation`. `profile` stores business metadata (VAT and Registration numbers). `address` stores physical location data.
- **staff:** Users who work for the organisation. They are linked to both an `organisation` and a `role` to dictate their permissions.
- **api\_key:** Stores hashed API keys for programmatic access, linked directly to an `organisation`.

## Module 3: Curriculum & Content

This module represents the strict hierarchy of the learning materials.

- **course**: The top-level curriculum entity created by a `staff` member. It tracks metadata like audience, requirements, difficulty, and a unique URL `cou_slug`.
- **chapter**: A sub-division of a course. A course has a One-to-Many relationship with chapters.
- **chapter\_section**: The atomic unit of learning. This holds the actual content, media URLs, and duration. Chapters have a One-to-Many relationship with sections.
- **attachment**: Downloadable files or resources linked to a specific `chapter_section`.

## Module 4: Assessment Engine (Quizzes)

The quiz system is highly denormalized for flexible assignment.

- **quiz**: The parent assessment entity containing the title, passing score, and max attempt limits. It is linked to a `course`.
- **quiz\_question**: Individual questions belonging to a quiz, supporting types like `MULTIPLE_CHOICE` and `TRUE_FALSE`.
- **quiz\_question\_option**: The possible answers for a question, noting which one is correct via `qto_is_correct`.
- **chapter\_quizzes**: A Many-to-Many junction table. This allows a single `quiz` to be assigned to multiple `chapters` for reusability.

## Module 5: Student Tracking & Progress

This module tracks the end-users (learners) and their interaction with the content.

- **student**: The learner entity, strictly bound to a specific `organisation`.
- **student\_enrollment**: Represents a student's registration in a `course`. It tracks global course progress (`ste_total_progress`) and acts as a bookmark for where they left off (`ste_chapter_section_id`).
- **student\_progress**: Granular tracking. It links an enrollment to a specific `chapter_section` to record completion percentages of individual lessons.
- **student\_quizzes**: A table tracking which quizzes a student is explicitly assigned or allowed to take.
- **student\_quiz\_attempt**: Records the actual execution of a quiz. It stores the calculated `sqa_score`, whether they passed, and importantly, saves the raw student inputs in a JSON column (`sqa_submitted_answers_json`) for audit purposes.

## Module 6: Course Discussions (Q&A)

This allows for community interaction within the courses.

- **course\_question:** A question asked by a `student` regarding a specific course (and optionally a specific `section`).
  - **course\_question\_reply:** Responses to a question. This utilizes a polymorphic relationship design. A reply can be authored by either a `student` or `staff` member, enforced at the database level by a `CHECK (cqr_student_id IS NOT NULL OR cqr_staff_id IS NOT NULL)` constraint.
- 

## 3. Order of Data Flow (The Happy Path)

To understand how data populates the system, follow this sequence:

1. **System Initialization:** The database is seeded with global `role` and `subscription_plan` records.
2. **Tenant Onboarding:** A business signs up, creating an `organisation`, `profile`, `address`, and an `organisation_subscription`.
3. **Staffing:** The Org Admin creates `staff` members, assigning them roles like `COURSE_EDITOR`.
4. **Content Creation:** A staff member creates a `course`, adds `chapters`, and populates them with `chapter_sections` and `attachments`.
5. **Assessment Creation:** The staff member builds a `quiz`, adds `quiz_questions` and `quiz_question_options`, and links the quiz to a chapter via `chapter_quizzes`.
6. **Student Registration:** A `student` is added to the organisation.
7. **Enrollment:** The student is linked to a course via `student_enrollment`.
8. **Learning:** As the student consumes content, records are upserted into `student_progress` for each section, and the total progress on the `student_enrollment` is updated.
9. **Testing:** The student attempts a quiz, resulting in a `student_quiz_attempt` record being saved with their score and raw JSON answers.